SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

2026 **BC. Marcel Soták**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav informatiky a matematiky (FEI)
Školiteľ:	Ing. Maroš Čavojský, PhD.

2026 **BC. Marcel Soták**

Slovenská technická univerzita v Bratislave
Ústav jadrového a fyzikálneho inžinierstva

Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025
Evidenčné číslo: FEI-XXXXX-YYYYYY

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Meno Priezvisko**
ID študenta:	012345
Študijný program:	jadrové a fyzikálne inžinierstvo
Študijný odbor:	elektrotechnika
Vedúci práce:	tituly, Meno Priezvisko, tituly
Vedúci pracoviska:	tituly Meno Priezvisko, tituly
Miesto vypracovania:	Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

1. Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
2. Napíšte návod na písanie záverečnej práce s použitím vytvorenej šablóny.
3. Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

1. Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf
2. Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecnych-prac-ich-bibliografickej-registracii-uchovavani-a-spristupnovani/.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice,* 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na http://doi.org/10.52036/1335793X.2023.3-4.10-22.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex--iso690/biblatex-iso690.pdf.

Termín odovzdania bakalárskej práce:	31. 05. 2025
Dátum schválenia zadania bakalárskej práce:	28. 02. 2025
Zadanie bakalárskej práce schválil:	tituly Meno Priezvisko, tituly – garant študijného programu

Poďakovanie

Týmto by som sa chcel úprimne poďakovať vedúcemu mojej diplomovej práce Ing. Marošovi Čavojskému, PhD. za jeho cenné rady, pomoc a vedenie počas písania tejto práce. Moje poďakovanie patrí taktiež mojej rodine za správnu výchovu, potrebunú k dopraqcovaniu až sem,mojej priateľke za psychickú podporu a poskytnutie príjemného prostredia pri písaní práce, ako aj Ministerstvu školstva za bezplatnú možnosť študovať na tejto fakulte.

Abstrakt

Táto diplomová práca sa zameriava na vývoj aplikácie, ktorá používateľom umožňuje jednoduché sociálne prepájanie prostredníctvom organizovania a vyhľadávania podujatí na základe ich polohy. Aplikácia podporuje vytváranie menších neformálnych stretnutí jednotlivcov so spoločnými záujmami, ako aj organizovanie väčších podujatí zo strany firiem a organizácií.

Cieľom práce bolo vytvoriť používateľsky prívetivú aplikáciu využívajúcu moderné postupy a technológie, ktorá zjednodušuje interakciu medzi používateľmi so spoločnými záujmami a podporuje živé stretnutia v reálnom čase.

Systém je navrhnutý ako Android aplikácia využívajúca GPS technológiu na zistenie polohy používateľa a poskytuje možnosti pre vytváranie a správu udalostí, pozývanie účastníkov a správu notifikácií. Aplikácia funguje aj na pozadí a je schopná posielať notifikácie. Využíva Firebase databázu pre ukladanie a synchronizáciu dát v reálnom čase, ako aj vlastný odporúčací systém pre navrhovanie udalostí na základe histórie používateľa a jeho záujmov. Aplikácia má taktiež implementovaný systém na detekciu nevhodných výrazov v názvoch a popisoch podujatí.

Výsledná aplikácia bola otestovaná na reálnych zariadeniach a hodnotená z hľadiska funkčnosti a použiteľnosti.

Kľúčové slová

záverečná práca, šablóna, LaTeX, formátovanie textu, citácie

Abstract

The manual for students of the Faculty of Electrical Engineering and Information Technology at Slovak University of Technology in Bratislava provides advice and guidelines on how to approach the formal aspects of writing a final thesis for university studies. The document can also serve as a template for the thesis in the LaTeX typesetting system. It covers in detail the rules for typesetting mathematical equations, numbering floating objects, and referencing them. It also thoroughly addresses the method of citing external literary sources.

Keywords

Final thesis, events, meeting, organizer, location, gps, kotlin, android

Obsah

Úvod **12**

1 Štruktúra záverečnej práce **13**
1.1 Asynchrónne spracovanie dát 13
1.2 Cloudové a mapové platformy 13

2 Vývoj mobilných aplikácií **14**
2.1 Prístupy k vývoju mobilných aplikácií 14
2.2 Technické obmedzenia a optimalizácia výkonu 14
2.3 Distribúcia a ekosystém platforiem 15
2.4 Toolkit Jetpack Compose 15
2.5 Programovací jazyk Kotlin 16

3 Architektonické vzory a princípy návrhu **18**
3.1 Základné princípy moderného návrhu 18
3.2 Porovnanie architektonických vzorov 18
3.3 Odporúčaná viacvrstvová architektúra 19

4 Moderné používateľské rozhranie s Jetpack Compose **20**
4.1 Princípy deklaratívneho návrhu 20
4.2 Dynamický životný cyklus: Rekompozícia a Stav 20
4.3 Vplyv na efektivitu vývoja 21
4.4 Firebase 21
4.4.1 Definícia a význam odporúčacích systémov 21
4.4.2 Obsahové filtrovanie (Content-Based Filtering) 21
4.4.3 Kolaboratívne filtrovanie (Collaborative Filtering) 22
4.4.4 Hybridné systémy 22
4.5 Techniky spracovania dát v odporúčacích systémoch 22
4.6 Metriky hodnotenia kvality odporúčaní 24
4.6.1 Úvod do odporúčacích systémov 25
4.7 Google Maps Platform 25
4.7.1 Maps SDK for Android 25
4.7.2 Places SDK for Android 25
4.8 Súčasný stav riešeneaj problematiky 26

Záver **27**

Literatúra **29**

Použitie nástrojov umelej inteligencie 29

A Algoritmus **30**

B Výpis dlhého kódu **31**

C Slovníček pojmov **35**

Zoznam značiek a skratiek

$\vec{a}, \boldsymbol{a}$ – zrýchlenie (m/s^2)

a – veľkosť zrýchlenia (m/s^2), mriežková konštanta (nm)

AC – striedavý prúd, z angl. *Accelerating Current*

AFM – mikroskop atómových síl, z angl. *Atomic Force Microscope*

AI – umelá inteligencia, z angl. *Artificial Intelligence*

ε_0, ε_0 – elektrická konštanta ($\varepsilon_0 = 8{,}85418782\,.\,10^{-12}\,\mathrm{m^{-3}kg^{-1}s^4A^2}$)

GAI – generatívna umelá inteligencia

λ – vlnová dĺžka svetla (nm)

LED – svetlo emitujúca dióda, z angl. *Light-Emitting Diode*

LLM – veľký jazykový model, z angl. *Large Language Model*

$\vec{F}, \boldsymbol{F}, F$ – vektor sily, veľkosť sily (N)

q – elektrický náboj (C)

$\boldsymbol{r}$, r – polohový vektor, veľkosť polohového vektora (m)

SEM – skenovací elektrónový mikroskop

TEM – transmisný elektrónový mikroskop

U – elektrické napätie (V)

Zoznam algoritmov

1 Vypočítaj $y = x^n$ 30

Zoznam výpisov kódov

1 Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting` 31

Úvod

Digitalizácia priniesla veľa výhod: online nákupy, objednanie jedla jedným klikom, stretnutia cez internet bez fyzickej prítomnosti druhej osoby a mnoho ďalších vecí, ktoré nám uľahčujú život. Ale nájdu sa v tomto svete aj nevýhody?

Čím viac nás vtiahne tento virtuálny svet, tým viac zabúdame na to, čo je realita. Nakoniec zistíme, že nekonečné scrollovanie na sociálnych sieťach nám neprinesie do života dlhodobú radosť, skôr nám ho urobí nudnejším. Robí nás pasívnejšími a závislejšími od technológií. Talenty, ktoré sme možno kedysi mali, sa postupne vytrácajú. A aj keby sme mali chvíľkovú potrebu s tým niečo spraviť, zabrzdí nás fakt, že tomu musíme venovať úsilie a čas.

Ale čo ak by existovala jednoduchá aplikácia, kde si človek môže nájsť ľudí so spoločnými záujmami, akcie, ktoré sa mu páčia, alebo objaviť nové hobby len preto, lebo vyzerajú zaujímavo na fotkách? Toto je dôvod, prečo som sa rozhodol vytvoriť túto mobilnú aplikáciu.

Cieľom tejto diplomovej práce je vytvoriť mobilnú aplikáciu pre systém Android, ktorá spája ľudí, pomáha organizátorom jednoducho vytvárať vlastné podujatia a pomáha používateľom nájsť zábavu, nové záujmy alebo len tak tráviť čas s priateľmi. Pri tejto práci využijeme množstvo postupov a technológií, ktoré nám umožnia vytvoriť funkčnú a používateľsky príjemnú aplikáciu.

V práci sa budeme venovať návrhu a implementácii tejto aplikácie. Využijeme databázy na ukladanie informácií o používateľoch a podujatiach a navrhneme odporúčací systém vzhľadom na používateľovu históriu. Taktiež navrhneme systém, ako zamedziť zneužívaniu aplikácie a ako zabezpečiť kontrolu nevhodného obsahu. Aplikácia bude mať jednoduché a intuitívne rozhranie, ktoré umožní používateľom ľahko sa orientovať a využívať všetky funkcie. Nakoniec budeme aplikáciu testovať a získavať spätnú väzbu od používateľov, aby sme ju mohli vylepšiť a prispôsobiť ich potrebám. Verím, že táto aplikácia bude užitočná pre mnoho ľudí, pomôže im nájsť nové záujmy a že jej finálna verzia prinesie do tohto sveta hodnotu.

1 Štruktúra záverečnej práce

Táto diplomová práca je rozdelená na niekoľko logických celkov, ktoré postupne opisujú cestu od teoretickej analýzy až po finálnu implementáciu a testovanie aplikácie.

1.1 Asynchrónne spracovanie dát

1.2 Cloudové a mapové platformy

2 Vývoj mobilných aplikácií

2.1 Prístupy k vývoju mobilných aplikácií

Podľa spoločnosti IBM možno proces vývoja mobilných aplikácií definovať ako tvorbu softvéru navrhnutého špecificky pre mobilné koncové zariadenia, pričom kľúčovým rozhodnutím je technológia implementácie. V súčasnosti sa vývojári rozhodujú primárne medzi natívnym a multiplatformovým prístupom, ktoré sa líšia výkonom a náročnosťou na zdroje [**pullur_medium_2025**]:

- **Natívne aplikácie:** Sú vyvíjané priamo pre konkrétnu platformu (Android, iOS) pomocou dedikovaných nástrojov. Ponúkajú najvyšší výkon a neobmedzený prístup k hardvéru, čo je kľúčové pre aplikácie náročné na systémové zdroje.
- **Webové mobilné aplikácie:** Ide o responzívne webové stránky využívajúce technológie HTML5, CSS a JavaScript, ktoré bežia v prehliadači zariadenia. Modernou formou sú tzv. *Progressive Web Apps* (PWA), ktoré umožňujú inštaláciu na domovskú obrazovku, avšak ich prístup k nízkoúrovňovým funkciám hardvéru (napr. pokročilý geofencing) je limitovaný.
- **Hybridné aplikácie:** Kombinujú predchádzajúce prístupy. Jadro aplikácie je písané vo webových technológiách, ktoré sú následne "zabalené"do natívneho kontajnera. Tento prístup uľahčuje multiplatformový vývoj, no prináša výkonnostnú réžiu spojenú s prekladom volaní medzi webovou a natívnou vrstvou.

2.2 Technické obmedzenia a optimalizácia výkonu

Vývoj pre mobilné platformy vyžaduje špecifický prístup k správe prostriedkov, ktoré sú v porovnaní s desktopovými systémami limitované. Moderný vývojár sa zameriava na tri kľúčové oblasti:

1. **Energetická a systémová efektivita:** Natívne aplikácie vykazujú vyššiu energetickú efektivitu, čo je kľúčové pri aplikáciách využívajúcich lokalizačné služby (GPS). Efektívna správa prostriedkov priamo ovplyvňuje výdrž batérie zariadenia [**pullur_medium_2025**].
2. **Cloudové spracovanie (*Cloud Offloading*):** Náročné výpočtové operácie, ako sú komplexné odporúčacie algoritmy, je podľa odporúčaní IBM vhodné delegovať na cloudové servery. Tým sa predchádza preťaženiu lokálneho hardvéru a nadmernému zahrievaniu zariadenia.

3. **Odozva používateľského rozhrania (UI):** Natívne frameworky (ako Jetpack Compose) dosahujú najlepšiu plynulosť animácií a rýchlosť odozvy na dotyk. Vďaka priamemu prístupu k vykresľovacím API systému je minimalizované riziko zasekávania rozhrania pri spracovaní dát na pozadí.

2.3 Distribúcia a ekosystém platforiem

Voľba operačného systému ovplyvňuje nielen technologický zásobník, ale aj stratégiu uvedenia na trh. Platforma Android, s približne 70 % trhovým podielom, je často preferovaná pre nasadenie minimálneho životaschopného produktu (MVP) kvôli flexibilnejším pravidlám v obchode Google Play. Naopak, ekosystém iOS vyžaduje striktnejšie dodržiavanie dizajnových a bezpečnostných štandardov, čo sa odzrkadľuje na vyššej miere udržania používateľov.

Avšak je dôležité poznamenať, že multiplatformové nástroje, ako sú Flutter a React Native, umožňujú súčasné nasadenie na oboch platformách, avšak s kompromisom v oblasti výkonu a optimalizácie zdrojov. Preto je kritické zhodnotiť požiadavky projektu a očakávania používateľov pri výbere vhodného prístupu k vývoju mobilnej aplikácie. toto su vysledky podla https://medium.com/@riyaspullurofficial/native-vs-cross-platform--app-size-and-performance-comparison-550cb7f918b0

Tabuľka 1: Súhrnné porovnanie natívneho a multiplatformového vývoja (spracované podľa [**pullur_medium_2025**])

Metrika	**Native**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Rýchlosť štartu	**Najrýchlejšia**	Stredná	Pomalšia	Rýchla
Animácie	**Plynulé**	Plynulé	Riziko lagov	–
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Debugging	**Najlepšie nástroje**	Silné	Slabšie	–
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka
Optimalizácia zdrojov	**Pokročilá**	Zlepšuje sa	Základná	–

2.4 Toolkit Jetpack Compose

Jetpack Compose je odporúčaný moderný toolkit spoločnosti Google na budovanie natívneho používateľského rozhrania (UI). Zjednodušuje a urýchľuje vývoj UI na systéme Android tým, že umožňuje rýchlo oživiť aplikáciu s menším množstvom kódu, výkonnými nástrojmi a intuitívnym API v jazyku Kotlin.

Kľúčové vlastnosti Jetpack Compose zahŕňajú:

- **Menej kódu:** Robte viac s menším množstvom kódu a vyhnite sa celým triedam chýb. Kód je jednoduchý a ľahko udržiavateľný.
- **Intuitívnosť:** Stačí opísať vaše UI a Compose sa postará o zvyšok. Pri zmene stavu aplikácie sa používateľské rozhranie automaticky aktualizuje.
- **Zrýchlenie vývoja:** Je kompatibilný so všetkým vaším existujúcim kódom, takže ho môžete prijať kedykoľvek a kdekoľvek chcete. Umožňuje rýchlu iteráciu pomocou živých náhľadov (*Live Previews*) a plnú podporu v prostredí Android Studio.
- **Výkonnosť:** Compose umožňuje vytvárať atraktívne aplikácie s priamym prístupom k API platformy Android a so vstavanou podporou pre *Material Design*, tmavý režim, animácie a ďalšie funkcie. Rieši deklaratívne UI, prístupnosť (*Accessibility API*) a rozloženie (*Layout*), čím skracuje cestu medzi nápadom a finálnym produktom.

2.5 Programovací jazyk Kotlin

Kotlin je moderný programovací jazyk, ktorý je stručný, multiplatformový a plne interoperabilný s jazykom Java a inými jazykmi. Ide o staticky typovaný jazyk, ktorý využíva viac ako 60 % profesionálnych vývojárov pre Android, pretože pomáha zvyšovať produktivitu, spokojnosť vývojárov a bezpečnosť kódu.

Medzi jeho hlavné výhody patrí:

- **Expresívny a stručný:** Moderné funkcie jazyka Kotlin umožňujú zamerať sa na vyjadrenie myšlienok a písať menej redundantného kódu.
- **Bezpečnejší kód:** Vďaka správe nulových hodnôt (*Nullability*) priamo v systéme typov pomáha predchádzať chybám typu *NullPointerException*. Aplikácie pre Android používajúce Kotlin majú o 20 % nižšiu pravdepodobnosť pádu.
- **Základ pre Jetpack Compose:** Moderný toolkit Jetpack Compose je postavený na Kotline, čo umožňuje rýchlo vytvárať adaptívne UI pomocou výkonných a intuitívnych rozhraní.
- **Kotlin Multiplatform:** Umožňuje zdieľať kód naprieč platformami Android, iOS a desktopovými systémami.

Jetpack Compose je postavený priamo okolo jazyka Kotlin. V niektorých prípadoch Kotlin poskytuje špeciálne idiómy, ktoré uľahčujú písanie kvalitného kódu v rámci Compose.

3 Architektonické vzory a princípy návrhu

Úspešná mobilná aplikácia musí byť postavená na robustnej architektúre, ktorá zaručuje jej škálovateľnosť a udržateľnosť v ekosystéme rôznych zariadení (telefóny, tablety, ChromeOS či skladacie zariadenia). Podľa oficiálnej dokumentácie spoločnosti Google je základným cieľom architektúry definovať hranice medzi jednotlivými časťami aplikácie a jasne určiť ich zodpovednosť.

3.1 Základné princípy moderného návrhu

Pred definovaním konkrétnych vzorov je potrebné určiť pravidlá, ktoré musí moderná architektúra spĺňať pre zachovanie stability v prostredí s obmedzenými systémovými prostriedkami:

- **Separation of concerns (Oddelenie zodpovedností):** Kľúčový princíp, ktorý zakazuje sústredenie kódu v triedach *Activity* alebo *Fragment*. Tieto komponenty slúžia primárne ako hostitelia používateľského rozhrania, sú krátkodobé (*ephemeral*) a operačný systém ich môže kedykoľvek zničiť z dôvodu nedostatku pamäte alebo zmeny konfigurácie.
- **Drive UI from data models (Riadenie UI dátovými modelmi):** Používateľské rozhranie by malo byť postavené na dátových modeloch, ktoré sú nezávislé od životného cyklu UI prvkov. Perzistentné modely zabezpečujú, že aplikácia nestratí dáta pri reštarte procesu a zostáva funkčná aj pri nestabilnom pripojení k sieti.
- **Single Source of Truth (SSOT - Jednotný zdroj pravdy):** Každý dátový typ má priradeného svojho vlastníka, ktorý ako jediný môže dáta modifikovať. Zmeny sa následne propagujú prostredníctvom imutabilných typov, čo znižuje výskyt chýb a uľahčuje ladenie systému.
- **Unidirectional Data Flow (UDF - Jednosmerný tok dát):** Stav aplikácie prúdi smerom nadol (od SSOT k UI elementom) a udalosti/akcie prúdia smerom nahor. Tento vzor zaručuje konzistenciu dát v celom systéme.

3.2 Porovnanie architektonických vzorov

V histórii vývoja platformy Android sa vyvinulo niekoľko prístupov k organizácii kódu, ktoré reagovali na narastajúcu komplexnosť aplikácií:

1. **MVC (Model-View-Controller):** Tradičný model, kde Controller riadi tok dát. V Androide tento vzor zlyhával, pretože triedy *Activity* preberali úlohu controllera aj view súčasne, čo viedlo k vzniku ťažko udržiavateľného kódu.

2. **MVP (Model-View-Presenter):** Oddelil zodpovednosť medzi View a Presenter. Problémom zostala vysoká previazanosť a nutnosť manuálneho ošetrovania životného cyklu, čo spôsobovalo úniky pamäte (*memory leaks*).

3. **MVVM (Model-View-ViewModel):** Súčasný štandard. *ViewModel* uchováva stav aplikácie nezávisle od UI komponentov, čím rieši problém s reštartovaním aktivít a stratu dát pri zmene orientácie zariadenia.

3.3 Odporúčaná viacvrstvová architektúra

Moderná Android architektúra (MAA) definuje tri základné vrstvy:

- **Používateľská vrstva (UI Layer):** Zobrazuje dáta na obrazovke a reaguje na interakciu používateľa. Skladá sa z UI elementov (vytvorených v *Jetpack Compose*) a držiteľov stavu (*ViewModels*), ktorí transformujú biznis dáta do formy vhodné pre zobrazenie.

- **Dátová vrstva (Data Layer):** Obsahuje biznis logiku aplikácie – pravidlá, ako sa dáta vytvárajú, ukladajú a menia. Skladá sa z **repozitárov**, ktoré zapuzdrujú prístup k rôznym zdrojom dát (Firebase, lokálna pamäť).

- **Doménová vrstva (Domain Layer):** Voliteľná vrstva, ktorá zapuzdruje komplexnú biznis logiku využiteľnú viacerými ViewModelmi súčasne (tzv. *Use Cases*).

4 Moderné používateľské rozhranie s Jetpack Compose

Tradičný vývoj používateľského rozhrania v systéme Android sa dlhé roky spoliehal na imperatívny prístup pomocou XML šablón. S príchodom knižnice **Jetpack Compose** však nastal posun k **deklaratívnej paradigme**. Tento prístup zásadne mení spôsob, akým aplikácia synchronizuje dáta s vizuálnou reprezentáciou na obrazovke [**android_compose_docs**].

4.1 Princípy deklaratívneho návrhu

Podľa oficiálnej dokumentácie vývojár v Compose definuje rozhranie ako priamu funkciu stavu ($UI = f(state)$). Tento model prináša niekoľko kľúčových konceptov:

- **The Declarative UI:** Vývojár popisuje cieľový stav rozhrania, nie jednotlivé kroky k jeho dosiahnutiu. To eliminuje potrebu manuálnej manipulácie s prvkami (napr. `findViewById`).
- **Encapsulation (Zapuzdrenie):** Kompozitné funkcie (*Composables*) sú samostatné jednotky, ktoré v sebe zapuzdrujú logiku zobrazenia aj správania, čím zvyšujú modularitu kódu.
- **Composition over Inheritance:** Na rozdiel od staršieho systému, kde každý prvok musel dediť od triedy `View`, Compose stavia na skladaní komponentov. To umožňuje väčšiu flexibilitu pri tvorbe komplexných prvkov, ako sú karty podujatí v odporúčacom systéme.

4.2 Dynamický životný cyklus: Rekompozícia a Stav

Kľúčovým mechanizmom, ktorý zabezpečuje responzivitu aplikácie, je **rekompozícia**. Ide o proces inteligentného prekresľovania, kedy framework opätovne spúšťa len tie časti kódu, ktorých stav (*state*) sa zmenil.

- **State Hoisting:** Ide o vzor návrhu, kde sa stav presúva do vyššej zložky (caller), čím sa kompozitná funkcia stáva *stateless* (bezstavová). To uľahčuje testovateľnosť a znovupoužiteľnosť komponentov.
- **Inteligentná optimalizácia:** Dokumentácia uvádza, že rekonštrukcia môže prebiehať veľmi často (napríklad pri animáciách), preto sú funkcie v Compose navrhnuté tak, aby boli rýchle a nemali vedľajšie účinky (*side-effects*).

4.3 Vplyv na efektivitu vývoja

Prechod na Jetpack Compose podľa Google znižuje objem kódu potrebného na tvorbu UI o približne 40 %. V kontexte tejto práce to umožňuje rýchlu iteráciu návrhu mapových prvkov a dynamických zoznamov, pričom je zachovaná vysoká miera bezpečnosti voči chybám v životnom cykle aplikácie.

4.4 Firebase

Firebase predstavuje súbor nástrojov určených na budovanie, zlepšovanie a rozširovanie aplikácií. Poskytované služby pokrývajú veľkú časť funkcionalít, ktoré by si vývojári za bežných okolností museli programovať sami, hoci sa primárne chcú sústrediť na samotný používateľský zážitok (UX). Tieto služby zahŕňajú analytiku, autentifikáciu, databázy, konfiguráciu, úložiská súborov, push správy a mnohé ďalšie. Všetky služby sú hostované v cloude a vyžadujú si od vývojára minimálne úsilie pri ich škálovaní.

4.4.1 Definícia a význam odporúčacích systémov

S prudkým rozvojom internetových technológií a inteligentných mobilných zariadení čelí moderný používateľ fenoménu informačného preťaženia. V prostredí s takmer neobmedzeným množstvom digitálneho obsahu a služieb sa odporúčacie systémy (angl. *Recommender Systems*) stali kľúčovým nástrojom na personalizovanú filtráciu informácií.

Ich primárnou úlohou je predpovedať preferencie používateľa a zredukovať rozsiahly priestor dostupných položiek na úzky výber relevantných možností, čím sa zvyšuje celková spokojnosť a efektivita interakcie s platformou. Súčasný výskum sa čoraz viac orientuje na spracovanie implicitných dát. Ide o behaviorálne vzorce, ako sú kliknutia (*click-stream*), história navštívených miest či čas strávený interakciou s konkrétnym objektom [**electronics11010141**].

4.4.2 Obsahové filtrovanie (Content-Based Filtering)

Tento prístup je založený na analýze špecifických atribútov položiek. Systém identifikuje vlastnosti objektov (napr. textové informácie, kategórie alebo tagy), o ktoré používateľ v minulosti prejavil záujem, a na ich základe generuje odporúčania pre podobné položky [**electronics11010141**].

- **Princíp:** Analýza atribútov a metadát položiek (napr. žáner, kategória) s cieľom nájsť podobnosť s preferenciami používateľa.
- **Využitie:** Efektívne najmä pri službách pracujúcich s textovými dátami, hudbou alebo e-commerce [**electronics11010141**].

- **Limity:** Nízka miera diverzity (angl. *serendipity*) – používateľovi nie sú ponúkané nové, neobjavené kategórie obsahu, čo vedie k efektu informačnej bubliny.

4.4.3 Kolaboratívne filtrovanie (Collaborative Filtering)

Kolaboratívne filtrovanie predpovedá vkus používateľa na základe analýzy správania a hodnotení celej komunity používateľov (tzv. „wisdom of the crowd“).

- **Memory-based:** Hľadá podobnosť priamo medzi používateľmi (*User-based*) alebo položkami (*Item-based*) pomocou algoritmov ako KNN alebo korelačných koeficientov.
- **Model-based:** Využíva pokročilé štatistické metódy a techniky strojového učenia, ako sú zhlukovanie (*Clustering*), *Singular Value Decomposition* (SVD) alebo *Principal Component Analysis* (PCA).
- **Hlavné nedostatky:** Zdroj [**electronics11010141**] definuje tri kritické obmedzenia:
  - *Cold Start:* Problém s odporúčaním položiek pre nových používateľov bez histórie interakcií.
  - *Sparsity (Riedkosť dát):* Situácia, keď je v systéme príliš málo hodnotení na to, aby bolo možné spoľahlivo vypočítať podobnosť.
  - *Gray Sheep (Sivé ovce):* Používatelia s veľmi unikátnym vkusom, ktorí nevykazujú podobnosť so žiadnou väčšou skupinou používateľov, čo sťažuje generovanie odporúčaní.

4.4.4 Hybridné systémy

Hybridné modely kombinujú viaceré techniky s cieľom eliminovať slabiny jednotlivých prístupov, najmä problém riedkosti dát (*sparsity*) a studeného štartu. Podľa Burkeho klasifikácie [**electronics11010141**] existuje sedem základných metód hybridizácie, ktoré sú uvedené v tabuľke 2. Hlavným cieľom väčšiny štúdií zaoberajúcich sa hybridnými modelmi po roku 2010 je integrácia tzv. vedľajších informácií (*side information*), ako sú sociálne väzby medzi používateľmi alebo kontextové dáta, ktoré dopĺňajú chýbajúce explicitné hodnotenia. Týmto spôsobom hybridné systémy dosahujú vyššiu presnosť a robustnosť v porovnaní so samostatnými modelmi.

4.5 Techniky spracovania dát v odporúčacích systémoch

Pre efektívne fungovanie vyššie spomínaných modelov sa využívajú špecifické techniky dolovania dát (Data Mining):

Tabuľka 2: Typy hybridných odporúčacích systémov podľa Burkeho [**electronics11010141**]

Metóda	**Princíp fungovania**
Weighted	Skóre z rôznych techník sa kombinuje pomocou numerických váh.
Switching	Systém prepína medzi technikami v závislosti od kontextu.
Mixed	Odporúčania z viacerých zdrojov sú zobrazené súčasne.
Feature Combination	Atribúty z jedného zdroja slúžia ako vstupné črty pre druhý model.
Feature Augmentation	Výstup jednej techniky sa stáva parametrom pre ďalšie spracovanie.
Cascade	Postupné zjemňovanie poradia (jeden systém vytvorí rebríček, druhý ho upresní).
Meta-level	Celý model vygenerovaný jednou technikou slúži ako vstup pre druhú.

1. **Text Mining:** Analýza neštruktúrovaných textov pomocou sémantickej analýzy a váhovania pojmov (napr. TF-IDF). Slúži na pochopenie obsahu podujatí a preferencií používateľa.

2. **K-Nearest Neighbor (KNN):** Algoritmus slúžiaci na klasifikáciu položiek alebo používateľov na základe ich blízkosti v metrickom priestore (geografickom alebo vkusovom).

3. **Clustering (Zhlukovanie):** Automatické rozdeľovanie dát do skupín so spoločnými črtami. Využíva sa najmä pri segmentácii používateľov alebo kategorizácii lokalít.

4. **Matrix Factorization:** Pokročilá technika rozkladu matíc, ktorá transformuje interakcie používateľov do priestoru latentných faktorov, čím rieši problém riedkosti dát (Sparsity).

5. **Neural Networks:** Modely hlbokého učenia schopné modelovať nelineárne vzťahy a spracovávať komplexné dáta zo sociálnych sietí či senzorov mobilných zariadení.

4.6 Metriky hodnotenia kvality odporúčaní

Na overenie úspešnosti algoritmov a kvantifikáciu kvality odporúčaní sa využívajú metriky založené na porovnaní predpovedí systému so skutočnými preferenciami používateľa. Základným nástrojom pre tento výpočet je tzv. konfúzna matica (*Confusion Matrix*), ktorá klasifikuje výsledky do štyroch kategórií podľa tabuľky 3 [**electronics11010141**].

Tabuľka 3: Konfúzna matica pre odporúčacie systémy [**electronics11010141**]

	Položka relevantná	**Položka nerelevantná**
Odporúčaná	True Positive (TP)	False Positive (FP)
Neodporúčaná	False Negative (FN)	True Negative (TN)

Na základe týchto hodnôt definujeme nasledujúce kvalitatívne indikátory:

- **Presnosť (Precision):** Pomer relevantných odporúčaných položiek k celkovému počtu odporúčaní. Vyjadruje mieru úspešnosti systému pri výbere položiek.

$$Precision = \frac{TP}{TP + FP}$$

- **Úplnosť (Recall):** Pomer úspešne odporúčaných položiek k celkovému počtu všetkých relevantných položiek, ktoré mal systém nájsť. Recall je identický s metrikou TPR (*True Positive Rate*) [**electronics11010141**].

$$Recall = \frac{TP}{TP + FN}$$

- **F-Measure:** Harmonický priemer presnosti a úplnosti. Táto metrika sa využíva najmä kvôli faktu, že medzi Precision a Recall existuje vzťah nepriamej úmery (*trade-off*).

$$F = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **Presnosť (Accuracy):** Pomer všetkých správnych predpovedí (TP + TN) k celkovému počtu prípadov. Zdroj [**electronics11010141**] však upozorňuje, že pri nevyvážených dátach (*unbalanced data*) môže byť táto metrika zavádzajúca.

- **RMSE (Root Mean Squared Error):** Index využívaný na hodnotenie presnosti predikcie číselného hodnotenia. Meria smerodajnú odchýlku rozdielov medzi predpovedaným a skutočným hodnotením.

- **ROC a AUC:** ROC krivka vizualizuje vzťah medzi mierou falošných pozitív (FPR) a mierou skutočných pozitív (TPR). Keďže krivka je grafické vyjadrenie,

pre kvantitatívne porovnanie sa využíva plocha pod touto krivkou – **AUC** (*Area Under the Curve*). Hodnota AUC nad 0,8 sa považuje za indikátor vysokej presnosti modelu [**electronics11010141**].

4.6.1 Úvod do odporúčacích systémov

4.7 Google Maps Platform

V rámci geolokačných služieb aplikácia využíva dva kľúčové komponenty platformy Google Maps, ktoré umožňujú vizualizáciu dát a interakciu s konkrétnymi miestami.

4.7.1 Maps SDK for Android

Maps SDK for Android umožňuje pridávať mapové podklady do aplikácie (vrátane Wear OS) s využitím údajov Google Maps, zobrazenia máp a reakcií na gestá. SDK poskytuje nástroje na vkladanie doplňujúcich informácií o polohách a podporuje interakciu používateľa prostredníctvom značiek (*Markers*), polygónov a iných prekrytí (*Overlays*).

Platforma plne podporuje programovacie jazyky Kotlin a Java, pričom poskytuje dodatočné knižnice a rozšírenia pre pokročilé funkcie a moderné programovacie techniky.

4.7.2 Places SDK for Android

Places SDK for Android umožňuje vytvárať aplikácie, ktoré kontextuálne reagujú na okolité podniky a iné miesta v blízkosti zariadenia. Tento nástroj dopĺňa základné geografické služby systému Android o bohatú databázu miest, ktoré majú pre používateľa konkrétny význam.

Kľúčové funkcie a rozhrania zahŕňajú:

- **Programový prístup k databáze:** SDK poskytuje prístup k informáciám o miestnych podnikoch, bodoch záujmu a aktuálnej polohe zariadenia.
- **Autocomplete:** Ponúka predpripravené komponenty (*Widgets*) na predpovedanie názvov miest v reakcii na vyhľadávacie dopyty používateľa.
- **Definícia miesta (*Place*):** Miesto je v rámci API reprezentované rozhraním *Place*. Definujeme ho ako fyzický priestor s názvom (napr. firmy, pamiatky, geografické polohy). Objekt *Place* obsahuje údaje ako názov, adresa, ID miesta, geografická poloha, telefónne číslo, typ miesta či webová adresa.

Podobne ako Maps SDK, aj tento nástroj plne podporuje jazyky Kotlin a Java vrátane rozšírení pre pokročilý vývoj.

4.8 Súčasný stav riešeneaj problematiky

Firebase Authentication — prihlasovanie používateľov Firebase Firestore — kde sú uložené eventy a história Firebase Storage — ak máš fotky eventov Hilt / Dependency Injection — ak používaš Coroutines / Flow — asynchrónne operácie, dôležité pre moderný Android

Záver

„V tejto príručke sme sa snažili poskytnúť praktické pokyny pre vypracovanie záverečných prác na Fakulte elektrotechniky a informatiky STU v Bratislave. Obsahuje dôležité informácie o formálnej stránke dokumentu, ktoré sú nevyhnutné pre zabezpečenie jasnej štruktúry a správneho formátovania práce. Naša príručka kladie dôraz na dodržiavanie základných pravidiel, ktoré pomáhajú minimalizovať chyby a zlepšujú celkovú kvalitu predkladaných dokumentov.

Rovnakú pozornosť venujeme aspektom, ako sú formátovanie textu, zarovnávanie, riadkovanie a použitie preddefinovaných štýlov, ktoré prispievajú k profesionálnemu vzhľadu dokumentu. Dôkladné spracovanie matematických rovníc a grafov je ďalším dôležitým prvkom, ktorý môže ovplyvniť hodnotenie práce. Práca s týmito aspektmi zaisťuje, že obsah je prehľadný a presne odráža zamýšľané myšlienky.

V neposlednom rade sa naša príručka zameriava na význam citovania externých zdrojov, čo je zásadné pre zachovanie integrity a etických štandardov pri písaní akademických textov. Správne citovanie nielenže posilňuje argumentáciu, ale tiež pomáha predchádzať plagiátorstvu, ktoré môže mať vážne následky na akademickú kariéru.

Dodržiavanie týchto pokynov umožňuje vyhnúť sa bežným chybám a zvyšuje kvalitu záverečných prác, čím sa zlepšuje celkový dojem z akademických schopností. Dobre vypracovaná práca môže viesť k pozitívnemu hodnoteniu a uznaniu zo strany hodnotiteľov, čo je cieľom každého študenta.

Celkovo je príručka navrhnutá ako cenný nástroj na podporu pri písaní záverečných prác. Poskytnuté informácie a odporúčania prispejú k dosiahnutiu akademických cieľov a pomôžu vypracovať kvalitné a odborné dokumenty, ktoré budú spĺňať štandardy fakulty.“

(ChatGPT 3, 2024)

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Worde. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, neprípustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

Použitie nástrojov umelej inteligencie

OpenAI (2025), ChatGPT 4o, časť **??**, generovanie vzorca záznamu použitia AI.

OpenAI (2024), ChatGPT 3, Záver, generovanie textu.

OpenAI (2025), ChatGPT 4o, dodatok B, generovanie kódu programu.

OpenAI (2024), ChatGPT 3, dodatok C, generovanie zoznamu.

Dodatok A: Algoritmus

Algoritmus 1 Vypočítaj $y = x^n$

Require: $n \geq 0 \vee x \neq 0$

Ensure: $y = x^n$

```
y ⇐ 1
if n < 0 then
    X ⇐ 1/x
    N ⇐ −n
else
    X ⇐ x
    N ⇐ n
end if
while N ≠ 0 do
    if N is even then
        X ⇐ X × X
        N ⇐ N/2
    else {N is odd}
        y ⇐ y × X
        N ⇐ N − 1
    end if
end while
```

Dodatok B: Výpis dlhého kódu

Dlhšie kódy, prípadne kompletné programy vypisujeme do dodatkov práce. Nasledujúci program v jazyku Python vygeneruje súbor `thesis.tex`, ktorý je funkčne totožný s hlavným súborom tohto dokumentu. Stačí vyplniť prehľadný formulár pri volaní funkcie `generate_thesis_tex` na konci kódu a získate hlavný súbor vašej práce.

```
def generate_thesis_tex(
    author,
    reg_nr,
    title,
    title_en,
    day,
    month,
    year,
    keywords,
    keywords_en,
    study_programme,
    study_programme_en,
    study_field,
    study_field_en,
    training_workplace,
    training_workplace_en,
    supervisor,
    consultant=None,
    thesis_type="bp",
    language="sk",
):

    tex_class_option = thesis_type if thesis_type in ["bp", "dp"] else "bp"
    tex_language_option = "en" if language == "en" else ""

    if tex_language_option:
        document_class = f"\\documentclass[{tex_class_option},{tex_language_option
    }]{{FEIstyle}}"
    else:
        document_class = f"\\documentclass[{tex_class_option}]{{FEIstyle}}"

    tex_content = f"""
{document_class}

% Author of the thesis
\\FEIauthor{{{author}}}
```

```
% Evidence number
\\FEIregNr{{{reg_nr}}}

% Title
\\FEItitle{{{title}}}
\\FEItitleEn{{{title_en}}}

% Date
\\FEIdate{{{day}}}{{{month}}}{{{year}}}

% Keywords
\\FEIkeywords{{{keywords}}}
\\FEIkeywordsEn{{{keywords_en}}}

% Further details
\\FEIstudyProgramme{{{study_programme}}}
\\FEIstudyProgrammeEn{{{study_programme_en}}}
\\FEIstudyField{{{study_field}}}
\\FEIstudyFieldEn{{{study_field_en}}}
\\FEItrainingWorkplace{{{training_workplace}}}
\\FEItrainingWorkplaceEn{{{training_workplace_en}}}

% Supervisor
\\FEIsupervisor{{{supervisor}}}
"""

    if consultant:
        tex_content += f"\n\\FEIconsultant{{{consultant}}}\n"

    tex_content += f"""

% Bibliography database file
\\bibliography{{includes/bibliography.bib}}

\\begin{{document}}

\\frontmatter
\\FEIpdfInfo
\\FEIcover
\\FEItitlePage
\\includepdf[pages=-]{{includes/assignment.pdf}}
\\FEIthanks{{includes/thanks}}
\\FEIabstract{{includes/abstract}}
\\FEIabstractEn{{includes/abstractEN}}
```

```
\\FEIcontent
\\FEImanualListOfGlossaries{{includes/manual_glossary}}
\\FEIlistOfAlgorithms
\\FEIlistOfListings

\\mainmatter
\\FEIintroduction{{includes/introduction}}
\\FEIcore{{includes/core}}
\\FEIconclusion{{includes/conclusion}}
\\FEIbibliography
\\FEIaiDeclaration{{includes/ai_declaration}}
\\backmatter
\\FEIappendix{{Algoritmus\\label{{att:algorithms}}}}{{includes/attachmentA}}
\\FEIappendix{{Výpis dlhého kódu\\label{{att:listings}}}}{{includes/attachmentB}}
\\FEIappendix{{Slovníček pojmov\\label{{att:dictionary}}}}{{includes/attachmentC}}

\\end{{document}}
"""

    with open("thesis.tex", "w", encoding="utf-8") as file:
        file.write(tex_content)

    print("Súbor thesis.tex bol úspešne vygenerovaný.")


# Testovacie volanie funkcie
generate_thesis_tex(
    author="RNDr. Juraj Chlpík, PhD.",
    reg_nr="FEI-xxxx-xxxx",
    title="Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme
    LaTeX",
    title_en="Extended thesis template at FEI STU in Bratislava in LaTeX system",
    day="31",
    month="12",
    year="2024",
    keywords="záverečná práca, šablóna, LaTeX, formátovanie textu, citácie",
    keywords_en="Final thesis, template, LaTeX, text formatting, citations",
    study_programme="názov študijného programu",
    study_programme_en="Study program in English",
    study_field="názov študijného odboru",
    study_field_en="Field of the study in English",
    training_workplace="Názov školiaceho pracoviska",
    training_workplace_en="Training Work place",
    supervisor="tituly Meno Priezvisko, tituly",
    consultant="",
```

```
    thesis_type="bp",  # Možnosti: "bp", "dp"
    language="sk",  # Možnosti: "sk" (slovenčina - predvolená), "en" (angličtina)
)
```

Výpis kódu 1: Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting`

Dodatok C: Slovníček pojmov

Abstrakt – Krátke zhrnutie hlavných bodov dokumentu, vrátane cieľov, metodológie a výsledkov.

Bod (pt) – Jednotka merania veľkosti písma; 1 bod sa rovná približne 0,352 mm.

Bold – Písmo, ktoré je tučné, používané na zvýraznenie dôležitých častí textu a v nadpisoch.

Citácia – Uvedenie zdroja informácií v texte, ktoré umožňuje identifikáciu pôvodného materiálu.

Font – Typ písma, ktorý sa používa na zobrazenie textu. Rôzne fonty majú odlišný štýl a vzhľad.

Graf – Vizualizácia údajov, ktorá zobrazuje vzťah medzi premennými.

Kapitola – Hlavná časť dokumentu, ktorá sa zaoberá konkrétnou témou a je zvyčajne označená číslom alebo názvom.

Korektúra – Proces revízie a opravy textu s cieľom odstrániť chyby.

Kurzíva – Typ písma, ktorý je sklonený doprava a používa sa na zvýraznenie textu, názvov alebo cudzích výrazov.

Metodológia – Opis metód a techník použitých pri výskume alebo analýze.

Normostrana – Základná jednotka pre meranie dĺžky textu, ktorá zvyčajne obsahuje 1 800 znakov vrátane medzier.

Obrázok – Grafický prvok alebo fotografia, ktorá ilustruje text.

Odsadenie – Vzdialenosť prvého riadku odseku od okraja stránky.

Odsek – Základná jednotka textu, ktorá obsahuje súvislé myšlienky. V LaTeX-u sa odseky oddeľujú prázdnym riadkom v zdrojovom texte.

Plagiátorstvo – Prezentovanie cudzej práce alebo myšlienok ako vlastných bez riadneho citovania.

Rez písma – Variant typu písma, ktorý určuje štýlové úpravy, ako sú normálny, tučný (bold), kurzíva (italic) alebo podčiarknutie.

Riadkovanie – Vzdialenosť medzi jednotlivými riadkami textu.

Sadzba – Proces usporiadania textu a grafiky na stránke.

Súbor – Elektronický dokument, ktorý obsahuje text alebo dáta.

Šablóna – Prednastavené formátovanie pre rôzne typy textu, ktoré zabezpečuje jednotný vzhľad.

Tabuľka – Usporiadanie údajov do riadkov a stĺpcov.

Titulný list – Prvá stránka dokumentu, ktorá obsahuje základné informácie, ako je názov práce a meno autora.

Typografia – Umenie a technika usporiadania textu tak, aby bol esteticky príťažlivý a čitateľný.

Úvod – Prvá časť dokumentu, ktorá predstavuje tému a ciele výskumu.

Vedecká práca – Akademický dokument, ktorý prezentuje výsledky výskumu, analýzy alebo diskusie o špecifickej téme.

Zdroje primárne a sekundárne – Primárne zdroje sú originálne dokumenty; sekundárne zdroje interpretujú alebo analyzujú primárne zdroje.